2026四川省大学生程序设计竞赛题解

Sichuan Provincial Collegiate Programming Contest 2026 Tutorial

南京大学出题组

出题组名单(按照姓名字典序排序)

Zida Chen (chen_zida)

Lin Li (LingLuo_Official)

Ruimin Zhang (paulzrm)

Yukuan Zhu (HugeWide)

赛前估计题目难度

Easy	D H G
Medium	E F C L
Hard	K I A
Very Hard	B
Impossible	J

D. 那一天的回文字符串

题目描述

给定一个只由小写英文字母组成的字符串s。
一次操作中，可以分别任意重排所有奇数下标位置上的字符和所有偶数下标位置上的字符。
问是否可以经过这样的重排，使得字符串s 变成一个回文串。

数据范围：测试组数t 满足$1 \le t \le 100$，字符串长度$|s|$ 满足$1 \le |s| \le 100$。
出题人：Ruimin Zhang (paulzrm)

D. 那一天的回文字符串

关键观察

重排操作不会改变字符所在位置的奇偶性。
因此只需要分别统计：

- 奇数下标位置中每个字符的出现次数$cnt_1(c)$；
- 偶数下标位置中每个字符的出现次数$cnt_0(c)$。

然后考虑回文串中对称位置i 和$n+1-i$ 的下标奇偶性。

D. 那一天的回文字符串

分类讨论

若n 为偶数，则i 与$n+1-i$ 奇偶性不同。
每一对对称位置需要一个奇数下标字符和一个偶数下标字符相等，所以要求：

$$cnt_0(c) = cnt_1(c) \qquad \text{for all } c.$$

若n 为奇数，则i 与$n+1-i$ 奇偶性相同。
偶数下标位置必须全部两两配对，奇数下标位置允许一个字符作为中心，因此要求：

$$\#\{c \mid cnt_0(c) \text{ 为奇数}\} = 0,$$
$$\#\{c \mid cnt_1(c) \text{ 为奇数}\} \le 1.$$

时间复杂度为$O(|s|)$。

H. 最大权独立集问题

题目描述

给定n 个点，每个点有一个整数权值W_i。
定义两点i 和j 之间存在一条无向边，当且仅当$W_i \oplus W_j$（其中$\oplus$ 表示按位异或运算）的二进制表示中，1 的个数为奇数。
求最大权独立集的点权和。

数据范围：$1 \le T \le 10^5$，$1 \le n \le 5 \times 10^5$，$1 \le W_i \le 5 \times 10^5$。
出题人：Lin Li (LingLuo_Official)

H. 最大权独立集问题

性质

根据异或运算的性质，对于任意非负整数A 和B，有

$$\text{popcount}(A \oplus B) \equiv \text{popcount}(A) + \text{popcount}(B) \pmod 2$$

又节点i 和j 之间有边当且仅当$\text{popcount}(W_i \oplus W_j)$ 为奇数，结合上述性质，有：

$$\text{popcount}(W_i) + \text{popcount}(W_j) \equiv 1 \pmod 2$$

即当且仅当W_i 和W_j 的二进制中1 的个数的奇偶性不同，两点之间才会连边。

H. 最大权独立集问题

解法

根据上述结论，我们可以将所有的点分成两个集合：

- 集合S_0：包含所有$\text{popcount}(W_i)$ 为偶数的点。
- 集合S_1：包含所有$\text{popcount}(W_i)$ 为奇数的点。

由连边条件可知：集合S_0 和S_1 内部的点之间奇偶性相同，没有任何边。集合S_0 中的任意一点与集合S_1 中的任意一点奇偶性不同，必然存在一条边。因此这个图实际上是一个完全二分图。
因此，图的任何一个独立集，要么完全包含在S_0 中，要么完全包含在S_1 中。只需要计算出S_0 中所有点的权值之和Sum_0 和S_1 中所有点的权值之和Sum_1，答案即为$\max(\text{Sum}_0, \text{Sum}_1)$。时间复杂度$\mathcal{O}(n)$。

G. 禁忌教典的消失咒文

题目描述

给定序列$a_1, a_2, \ldots, a_n$。对于序列$b_1, b_2, \ldots, b_m$，定义能量：

$$E(b) = \sum_{i=1}^{m} (-1)^{i+1} b_i.$$

需要删除一个非空连续区间$[l, r]$，剩余元素按原相对顺序拼接。求有多少个区间使得删除后能量等于k。

数据范围：$1 \le n \le 10^6$，所有测试数据的n 之和不超过10^6。
出题人：Ruimin Zhang (paulzrm)

G. 禁忌教典的消失咒文

转化

定义带符号前缀和：

$$P_i = \sum_{t=1}^{i} (-1)^{t+1} a_t, \qquad P_0 = 0.$$

考虑删除$[l, r]$，令$i = l - 1$，$j = r$。
若$j - i$ 为偶数，后半段符号不变，删除后能量为：

$$P_i + P_n - P_j.$$

若$j - i$ 为奇数，后半段符号翻转，删除后能量为：

$$P_i + P_j - P_n.$$

因此只需要统计满足条件的二元组(i, j)，其中$0 \le i < j \le n$。

G. 禁忌教典的消失咒文

计数

删除后能量等于k。
若$j-i$ 为偶数，则i,j 同奇偶，需要:

$$P_j - P_i = P_n - k.$$

若$j-i$ 为奇数，则i,j 异奇偶，需要:

$$P_i + P_j = P_n + k.$$

从左到右枚举j，用两个哈希表分别维护此前偶数下标和奇数下标的P_i 出现次数。
对于当前j，查询同奇偶中的$P_i = P_j - (P_n - k)$，以及异奇偶中的$P_i = P_n + k - P_j$，累加答案后再加入P_j。
时间复杂度为$O(n)$ 或$O(n \log n)$，空间复杂度为$O(n)$。

E. 永恒的奥古斯都

题目描述

有一棵n 个点的树T，树的根节点为1。初始时点i 有颜色c_i（$0 \le c_i \le 1$）。进行若干次操作，每次操作选择一个满足$c_u = 0$ 的节点u，对u 子树内的所有点v 执行$c_v \leftarrow 1 - c_v$。给定最后得到的树T' 和每个点的颜色c'_i，求有多少种不同的可能初始状态T。答案对998244353 取模。

数据范围：$1 \le n \le 2 \times 10^5$。

出题人：Lin Li (LingLuo_Official)

E. 永恒的奥古斯都

转化

考虑树上差分。设$d_u = c_u \oplus c_{p_u}$（其中p_u 为u 的父亲，根节点的父节点视作状态恒为0）。
对节点u 的子树进行操作时，子树内部的边异或值都不变，只有u 与p_u 的颜色相对关系改变。因此对u 的子树翻转等价于将d_u 取反。
于是可以发现对于任意的终态，任何一个合法的初始状态c_0 与操作节点集合U 构成一一映射。问题转化为求合法操作集合U 的数量。

E. 永恒的奥古斯都

结论

操作集合U 合法，当且仅当：对于集合U 中任意一个没有真祖先在U 中的极小节点u，其最终状态c'_u 为1。

证明

若u 是U 中的极小节点，则在其祖先链上没有任何操作。在操作u 之前，u 的颜色必须为0。操作后u 颜色翻转为1。由于u 的真后代被操作时，翻转的子树一定不包含u，故此后u 的颜色不再改变。因此终态中c'_u 必须为1。而一旦满足这个条件，子树内的其余点一定可以通过调整操作顺序完成构造。

E. 永恒的奥古斯都

计数

于是可以写出DP：设$f(u)$ 表示以u 为根的子树中，在假设u 的所有祖先均不在U 中的前提下，选择合法的操作子集U 的方案数。
分两种情况讨论：

- 若$u \notin U$：u 的各个子节点v 依然满足没有祖先在U 中的条件，各个子树独立。方案数为$\prod_{v \in \text{child}(u)} f(v)$。
- 若$u \in U$：u 此时属于U 中的极小节点，必须满足$c'_u = 1$。此时u 的所有真后代已经拥有一个在U 中的祖先（即u），不再受极小节点条件的限制。因此u 子树内剩余的$\text{size}(u) - 1$ 个节点均可自由选择是否加入U。方案数为$2^{\text{size}(u)-1}$。

据此容易完成DP，时间复杂度$\mathcal{O}(n)$。

F. 交换余生

题目描述

给定一个长为n 的序列a。判断是否存在序列b 满足：

- $S(a) = S(b)$，其中$S(a)$、$S(b)$ 分别表示由序列a 和序列b 中所有元素构成的可重集；
- 不存在$1 \le i < n$ 满足$\gcd(b_1, \cdots, b_i) = \gcd(b_{i+1}, \cdots, b_n)$。

数据范围：$1 \le T \le 10$，$1 \le n \le 2 \times 10^5$，$1 \le a_i \le 10^{12}$。
出题人：Lin Li (LingLuo_Official)

F. 交换余生

设原序列的全局最大公约数为$g = \gcd(a_1, a_2, \ldots, a_n)$。
记前缀gcd 为$p_i = \gcd(a_1, \ldots, a_i)$，后缀gcd 为$q_{i+1} = \gcd(a_{i+1}, \ldots, a_n)$。

观察

$p_i = q_{i+1}$ 当且仅当$p_i = q_{i+1} = g$。

证明

显然p_i 和q_{i+1} 都是g 的倍数。如果$p_i = q_{i+1} = g'$，那么整个数组的$\gcd(p_i, q_{i+1})$ 也必然是h'。但整个数组的gcd 是g，所以必定有$g' = g$。

为了方便讨论，可以提前将所有的a_i 都除以g。此时问题等价于：全局gcd $= 1$，判断能否重排序列，使得不存在任何一个分割点i，满足$p_i = 1$且$q_{i+1} = 1$。
这里可以特判如果除以g 后序列中出现≥ 2 个1 则一定无解。

F. 交换余生

充要条件

合法的重排一定形如：

$$k_1P, \cdots, k_{i-1}P, (z), k_{i+1}Q, \cdots .k_nQ$$

其中左侧部分有公共质因子P，右侧部分有公共质因子Q，z 是唯一一个既不包含P 也不包含Q 的元素（这里z 也可以不存在）。

结论

一个序列存在合法重排，当且仅当能找到两个素数P, Q，使得整个序列中不能被P 且不能被Q 整除的元素不超过1 个。

F. 交换余生

枚举P

考虑如果有解，那么z 最多只有1 个。
则a_1, a_2 中必定包含答案需要的质因子P。
于是可以枚举a_1 的所有质因子P 进行验证。如果全部验证失败说明a_1 就是要找的z，此时再对a_2 的所有质因子做一次验证即可。

F. 交换余生

P 的验证

设所有不能被P 整除的元素组成集合W。如果$|W| \leq 1$ 一定合法。
否则W 中最多只能有一个数z 不能被Q 整除。以下有两种做法:

- 做法一：和找P 一样，w_1, w_2 中必定包含答案需要的质因子Q。于是再枚举质因子做一遍判断即可。
- 做法二：枚举z，只需要$W \backslash \{z\}$ 的gcd 大于1 则一定存在这样的Q。预处理出前后缀gcd 即可$\mathcal{O}(\log(A))$ 判断。

两种做法的时间复杂度分别为$\mathcal{O}\left(T\sqrt{A} + \omega(A)^2 \sum n\right)$
和$\mathcal{O}\left(T\sqrt{A} + \omega(A) \sum n \log(A)\right)$，并且实际上跑不满，均可通过本题。

C. 精灵对战

题目描述

现有n 种精灵，精灵之间存在克制关系，每种精灵可以派出多次。

给出每种精灵被哪些精灵克制（每种精灵最多被k 种精灵克制），和对手的出战序列m 只精灵，问击败对手所有精灵所需派出最少精灵数目。

交战规则如下（设当前A, B 精灵交战）：

- A 克制B，B 不克制A：B 被击败，A 存活；
- B 克制A，A 不克制B：A 被击败，B 存活；
- A, B 不相互克制：A, B 同归于尽；
- A, B 相互克制：B 被击败，A 存活。

数据范围：$1 \le n, m \le 10^5$，$1 \le k \le 30$；
出题人：Zida Chen (chen_zida)

C. 精灵对战

解法

考虑DP，设$f_{i,j}$ 表示：已击败对手前i 只精灵，且小z 当前在场的精灵为j 时的最小总花费。特殊的，$f_{i,0}$ 表示上一局两个精灵同归于尽。

进行分类讨论：

- **不换精灵：** 若j 能击败a_i，或者同归于尽，可直接转移：

$$f_{i,0/j} \leftarrow f_{i-1,j}$$

- **更换精灵：** 若当前没有精灵在场，或者在场的精灵被a_i 击倒，小z 必须派出一只新精灵击败对手或者同归于尽。有转移：

$$f_{i,0/j} \leftarrow 1 + \min_k f_{i-1,k}$$

最终答案在$f_{n,?}$ 中。暴力转移的总复杂度为$O(nm)$。

考虑进行优化。

C. 精灵对战

解法（优化）

仔细观察“不换精灵”的转移条件：精灵j 必须能击败a_i。

- 只有当j 克制a_i 时，j 才能在击败a_i 后继续存活。
- 每种精灵最多被k 种精灵克制。这意味着对于固定的对手精灵a_i，能克制它且存活下来的小z 精灵j 最多只有k 种。

所以，每一轮中有值的状态$f_{i,j}$ 极其稀疏：除了刚换上场的精灵外，不换精灵的有值状态j 最多只有k 个。

因此，我们不需要维护全部状态。只需利用滚动数组，每轮只转移那k 个满足克制关系的存活状态，并结合全局最小值处理转移。

时间复杂度成功优化至$O(mk)$。

本题验题人给出了多种做法，此处不再枚举。

L. 博德之跃 3

题目描述

给定一个由小写英文字母组成的字符串S，以及一个初始为空的字符串T。
你需要对S 执行若干次操作，直到它变为空字符串。每次你可以执行以下三种操作中的一种：

- 删除S 的第一个字符，并将其添加到T 的末尾。
- 删除S 的最后一个字符，并将其添加到T 的末尾。
- 选择非空串A, B 满足$S = ABA^R$，令$S = B$ 并将字符串A 添加到T 的末尾。

其中A^R 表示字符串A 的反串。
求在所有操作方案中，能得到的字典序最小的T。

数据范围：$1 \le T \le 10^5$，$1 \le |S| \le 5 \times 10^5$。
出题人：Lin Li (LingLuo_Official)

L. 博德之跃 3

观察

关键在于操作三。由于只能将字符添加到T 的末尾，操作三本质上是：当S 的首尾字符相同时，我们可以将首字符加入T，并免费删去尾部对应的相同字符。

长为$|A|$ 的操作三可以用$|A|$ 次单字符的操作三来替换，可以发现这样一定不劣。

L. 博德之跃 3

贪心策略

维护双指针L 和R。当$S_L \neq S_R$ 时取较小的一端。当$S_L = S_R = c$ 时，向内寻找第一个不等于c 的字符：
设左侧连续的c 有A 个，之后跟着的字符是x；右侧连续的c 有B 个，之后跟着的字符是y。比较x, y 与c 的字典序：

1. $x < c$ 且$y < c$：内部字符更优。为了尽早将x 或y 加入T，我们需要尽快消耗掉两端的c。此时贪心地使用$\min(A, B)$ 次操作三。
2. $x < c$ 且$y > c$：左侧需要尽早取较小的x，而右侧的c 比y 小，应当尽可能保留c。因此直接取走左侧的A 个c。
3. $x > c$ 且$y < c$：同理取走右侧的B 个c。
4. $x > c$ 且$y > c$：内部字符均劣于当前字符c，因此这里必须尽可能多地将c 加入T，即会将两端总计$A + B$ 个c 全部加入T。

特判一下所有字符都相同的情况即可，时间复杂度$\mathcal{O}(|S|)$。

K. 环基基环树

题目描述

给定一张无向连通简单图，它由某张基环树经过如下变换得到：
对于原图中的每个点x，选择一个整数$c_x \geq 3$，并将x 替换成一个长度为c_x 的简单环C_x。
对于原图中的每条边(u, v)，在环C_u 上选择一个点，在环C_v 上选择一个点，并在这两个点之间连一条边。
现在给出变换后的图，要求还原出任意一张与原图同构的基环树。

数据范围：$\sum n \leq 10^6$，$\sum m \leq 1333333$。
出题人：Ruimin Zhang (paulzrm)

K. 环基基环树

关键观察

变换后的图中有三类结构：

- 每个原图点扩展出来的简单环；
- 原图基环上的边，对应扩展环之间的非桥连接边；
- 原图基环外的树枝边，对应扩展环之间的桥。

因此，先找出所有桥并删去。
删去桥后：

- 每个扩展环内部的边仍然存在；
- 原图基环上的连接边仍然存在；
- 原图基环外的树枝连接边被删去。

K. 环基基环树

删去桥后的度数性质

考虑删去桥后的图。
对于某个扩展环C_x：

- 如果环上的一个点没有承担原图基环上的连接边，那么它只和环上的左右两个相邻点相连，度数为2；
- 如果环上的一个点承担了原图基环上的连接边，那么它除了两个环内邻点外，还会连接到别的扩展环，度数至少为3。

由于原图是基环树，每个原图点在基环上至多有两条非桥连接边。又因为$c_x \geq 3$，每个扩展环中一定存在度数为2 的点。

K. 环基基环树

缩回一个扩展环

在删去桥后的图中，如果一个点的度数为2，那么它的两条边一定都是所在扩展环的环内边。
因此，这个点和它的两个邻点一定来自原图中的同一个点，应当被缩成同一个点。
不断利用这个性质，同一个扩展环上的点会被全部合并。
同时，不同扩展环之间的连接边不会导致错误合并，因为连接边的端点在删去桥后的图中度数至少为3，不会作为度数为2 的点去合并两侧。

结论

按上述规则缩点后，每个缩点恰好对应原图中的一个点。

K. 环基基环树

还原原图边

完成缩点后，重新考虑变换后图中的每条边(u, v)。
设u 所在缩点为U，v 所在缩点为V。

- 若$U = V$，说明这条边是某个扩展环内部的边，不属于原图；
- 若$U \neq V$，说明这条边连接了两个扩展环，对应原图中的一条边。

因此，输出所有跨缩点的边，就得到还原后的基环树。
时间复杂度和空间复杂度均为$O(n + m)$。

I. 星系观测计划

题目描述

给定二维平面上不重合的n 个点，绕着原点$O(0,0)$ 匀速逆时针旋转。

用一个与坐标轴平行，周长最小的矩形覆盖这n 个点，设旋转角度为θ 时，矩形的周长为$P(\theta)$。

求下面表达式的值：

$$\lim_{T\to+\infty}\frac{1}{T}\int_0^T P(\theta)\mathrm{d}\theta$$

数据范围：$1\le n\le 2\times 10^5$；
出题人：Yukuan Zhu (HugeWide)

转化

首先，$P(\theta)$ 的周期为2π. 设$T = 2k\pi + t$，其中$k \in \mathbb{N}^*$ 且$0 \le t < 2\pi$，则：

$$\begin{aligned}\frac{1}{T}\int_0^T P(\theta)\mathrm{d}\theta &= \frac{1}{2k\pi + t}\left(k \times \int_0^{2\pi} P(\theta)\mathrm{d}\theta + \int_0^t P(\theta)\mathrm{d}\theta\right) \\ &= \frac{1}{2\pi + \frac{t}{k}}\left(\int_0^{2\pi} P(\theta)\mathrm{d}\theta + \frac{1}{k}\int_0^t P(\theta)\mathrm{d}\theta\right)\end{aligned}$$

因此有：

$$\lim_{T\to+\infty}\frac{1}{T}\int_0^T P(\theta)\mathrm{d}\theta = \frac{1}{2\pi}\int_0^{2\pi} P(\theta)\mathrm{d}\theta = \mathrm{E}[P(\theta)]$$

所求即为$P(\theta)$ 的期望。

下面展示两种计算$\mathrm{E}[P(\theta)]$ 的方法。

I. 星系观测计划

旋转的坐标变换公式

将点(x, y) 绕着原点逆时针旋转θ 角度得到的点坐标(x', y') 为：

$$x' = x\cos\theta - y\sin\theta, \quad y' = x\sin\theta + y\cos\theta$$

解法1

首先：

$$P(\theta) = 2\max_{i=1}^{n} x_i' + 2\max_{i=1}^{n}(-x_i') + 2\max_{i=1}^{n} y_i' + 2\max_{i=1}^{n}(-y_i')$$

考虑如下式子的求值方法（另外三个式子可通过变形用相同方法）：

$$\mathrm{E}\left[\max_{i=1}^{n} x_i'\right] = \mathrm{E}\left[\max_{i=1}^{n}(x_i\cos\theta - y_i\sin\theta)\right]$$

计算得出四个期望值以后，可以依据期望的线性性相加得出$\mathrm{E}[P(\theta)]$。

解法1（续）

不妨考虑$0 \le \theta \le \pi$ 的处理，另一个部分的处理可以通过将x, y 变号重新计算得到。即，令：

$$S([x],[y]) = \int_{\theta=0}^{\pi} \left(\max_{i=1}^{n} x_i' \right) = \int_{\theta=0}^{\pi} \left(\max_{i=1}^{n} (x_i \cos\theta - y_i \sin\theta) \right)$$

那么有：

$$\int_{\theta=\pi}^{2\pi} \left(\max_{i=1}^{n} x_i' \right) = S([-x],[-y])$$

$$\mathrm{E}\left[\max_{i=1}^{n} x_i \right] = \frac{S([x],[y]) + S([-x],[-y])}{2\pi}$$

解法1（续）

对于$0 < \theta < \pi$，有$\cos\theta > 0$，所以：

$$S([x],[y]) = \int_{\theta=0}^{\pi} \frac{1}{\cos\theta} \left(\max_{i=1}^{n} (x_i - y_i \tan\theta) \right) \mathrm{d}\theta$$

注意到这里的$x_i - y_i \tan\theta$ 实际上是（关于$\tan\theta$ 的）n 条直线，

可以求出对应的（上）凸包的各顶点，这本质上是max 得到结果的分段点。求得分段点之后，可以直接回到$S([x],[y])$ 进行分段积分求值。

这里求值的复杂度为$O(n \log n)$，瓶颈在于求解凸包。

注意

这个做法的精度损失较高。

解法2

由于旋转的对称性，矩形长的期望和宽的期望相等，因此：

$$\mathrm{E}[P(\theta)] = 4 \times \mathrm{E}\left[\max_{i=1}^{n} x_i' - \min_{i=1}^{n} x_i'\right] = 4\mathrm{E}[\Delta x']$$

其中$\Delta x'$ 是这n 个点在x 轴上投影区间的长度。

设这n 个点构成的凸包有m 条边，长度为$l_1, l_2, \cdots, l_m$，它们投影到x 轴长度为$l_1', l_2', \cdots, l_m'$。

将这n 个点的凸包所有边投影到x 轴，恰好覆盖了两次该区间。即上、下凸包的投影恰好覆盖各一次：

$$2\mathrm{E}[\Delta x'] = \sum_{i=1}^{m} \mathrm{E}[l_i']$$

I. 星系观测计划

解法2（续）

而某条边l_i 投影长度的期望为：

$$E[l_i] = \frac{1}{2\pi}\int_{\theta=0}^{2\pi} |l_i \cos\theta| \mathrm{d}\theta = \frac{l_i}{\pi}\int_{\theta=0}^{\pi} \cos\theta \mathrm{d}\theta = \frac{2}{\pi} \times l_i$$

因此：

$$\mathrm{E}[P(\theta)] = 4\mathrm{E}[\Delta x'] = 2\sum_{i=1}^{m} \mathrm{E}[l_i'] = \frac{4}{\pi}\sum_{i=1}^{m} l_i = \frac{4C}{\pi}$$

其中C 是凸包的周长。（事实上：该结论可以推广到任意凸图形）

做法的复杂度为$O(n \log n)$，为求凸包的复杂度。

fun fact

一定要求积分吗？不，$\frac{4}{\pi}$ 这个参数在样例中给出了。

A. 那一年的秘密基地

题目描述

给定一棵n 个点的树和一个访问顺序$a_1, a_2, \ldots, a_n$。
若选择点r 作为根，危险度$f_r(a)$ 为满足

$$1 \le i < j \le n, \qquad a_j \text{ 是} a_i \text{ 的祖先}$$

的二元组(i, j) 数量。
有q 次操作，每次交换a_x 和a_{x+1}。要求在初始状态以及每次操作后，输出

$$\min_{1 \le r \le n} f_r(a).$$

数据范围：$1 \le n, q \le 2 \cdot 10^5$。
出题人：Ruimin Zhang (paulzrm)

A. 那一年的秘密基地

核心观察

对两个点x, y，记$\mathrm{Side}(x, y)$ 为删去x 到y 路径上的第一条边后，包含x 的连通块。
当根为r 时，有

$$x \text{ 是} y \text{ 的祖先} \iff r \in \mathrm{Side}(x, y).$$

因此，一个有序点对的贡献等价于对某个连通块整体加1。

相邻交换

交换相邻两点$u = a_x$，$v = a_{x+1}$ 时，只有这一对的相对顺序改变，因此只需更新

$$\mathrm{Side}(u, v) \text{ 加1}, \qquad \mathrm{Side}(v, u) \text{ 加 } -1.$$

A. 那一年的秘密基地

连通块的区间表示

先以1 为根DFS，得到每个点的子树区间。
对于Side(x, y):

- 若x 不是y 的祖先，则

$$\text{Side}(x, y) = \text{subtree}(x).$$

- 若x 是y 的祖先，设z 为x 到y 路径上的下一个点，则

$$\text{Side}(x, y) = V \setminus \text{subtree}(z).$$

用线段树维护每个根r 当前的$f_r(a)$，支持区间加和全局最小值查询。

A. 那一年的秘密基地

初始值计算

设$\mathrm{pos}(v)$ 为点v 在排列中的位置。考虑点v 作为后出现的点时的贡献。若根在v 所在位置，则所有更早出现的点都会被v 祖先覆盖，先对全体根加

$$\mathrm{pos}(v) - 1.$$

对于删去v 后的每个连通块C，若根落在C 中，则C 内更早出现的点不再被v 覆盖，需要在C 上减去

$$|\{u \in C \mid \mathrm{pos}(u) < \mathrm{pos}(v)\}|.$$

这些连通块仍可表示为子树或子树补集。

A. 那一年的秘密基地

如何快速处理

所有形如

$$|\{u \in C \mid \mathrm{pos}(u) < \mathrm{pos}(v)\}|$$

的询问，可以离线完成。
按$\mathrm{pos}(v)$ 从小到大扫描，用树状数组在DFS 序上维护已经出现的点，即可查询任意子树中已经出现了多少点。
得到这些值后，将每个点v 的贡献用线段树区间加加入初始答案。

复杂度

初始建树$O(n \log n)$，每次交换只做常数次区间加，复杂度$O(\log n)$。
总时间复杂度:

$$O((n+q)\log n).$$

B. 括号序列

题目描述

下面给出一个可匹配的括号串T，求有多少个括号串S 满足以下条件：

1. S 不是空串；
2. S 是可匹配的；
3. S 字典序小于等于T；
4. S 的长度小于等于T 的长度。

数据范围：字符串T 的长度n 满足$2 \leq n \leq 10^6$，且n 是偶数；T 是可匹配的；

出题人：Yukuan Zhu (HugeWide)

B. 括号序列

转化

首先对该问题转化为二维网格的路径计数，从原点$O(0,0)$ 开始移动，左括号表示向x 轴正方向移动1 单位距离，右括号表示向y 轴正方向移动1 单位距离，上述条件可以转化为：

1. 移动的距离不为0；
2. 中途不可移动到$y = x$ 上方，并且终点在$y = x$ 上；
3. S 对应的移动路径是T 对应的移动路径的某个前缀；或者在S, T 对应的移动路径第一个分叉点，S 对应的路径朝着x 轴正方向移动，T 对应的路径朝着y 轴正方向移动；
4. 移动的总距离不能超过n。

B. 括号序列

子问题

求在二维网格中，从(a, b) 到(k, k)，满足如下条件的路径数量：

1. 每一步向着x 或y 轴正方向移动1 单位距离；
2. 中途不可移动到直线$y = x$ 上方；

子问题的条件对应了原问题的条件2。

子问题的解法

- 若$a > k$ 或$b > k$ 或$a < b$，不存在合法的路径；
- 若$a, b \le k$ 且$a \ge b$，考虑用所有路径数目减去不合法路径数目；后者即与直线$y = x + 1$ 相交的路径，可以将路径从第一处交点至终点(t, t) 沿着该直线进行翻转，得到(a, b) 到$(t - 1, t + 1)$ 的路径，这与不合法路径一一对应，因此子问题的答案为：

$$\binom{2k - a - b}{k - a} - \binom{2k - a - b}{k - a - 1}$$

B. 括号序列

解法

原问题约束较多，考虑按照条件3进行分类。

先对所有非空前缀S 检查是否符合要求。这部分复杂度是$O(n)$ 的。

剩余情况，S 不是T 的前缀，此时S 非空，仅受条件2、4约束。

设S 对应路径p_S，T 对应路径p_T，则p_S 与p_T 有分叉点，设第一个分叉点为P:

- 若P 处p_T 向x 轴正方向移动，不存在合法的p_S;
- 若P 处p_T 向y 轴正方向移动，此处的p_S 必然向x 轴正方向移动，后面的移动方案不受约束。

可以枚举p_S, p_T 的首个分叉点位置P，和终点(k, k)，规约到子问题处理。

子问题求解是$O(1)$ 的，但是有$O(n^2)$ 数目的子问题。

考虑从该方法出发进行优化。

B. 括号序列

解法（续）

令$m = \frac{n}{2}$ 为T 长度的一半。不妨假设在(a, b) 处S, T 对应的两条路径出现首次分叉，T 对应的路径移动到$(a, b+1)$。

考虑计算给定分叉点下的答案数目，可以枚举终点(k, k) $(1 \le k \le m)$，计算$(a+1, b)$ 到(k, k) 的路径数：

$$res = \sum_{k=1}^{m} \binom{2k-a-b-1}{k-a-1} - \binom{2k-a-1-b}{k-a-2}$$

令$f(i,j) = \sum_{k=0}^{j} \binom{i+2k}{k}$，则：

$$res = f(a+1-b, m-a-1) - f(a+3-b, m-a-2)$$

问题即为求$O(n)$ 个f 的点值。

B. 括号序列

解法（续）

对f 函数进行变形：

$$f(i,j+1)=\sum_{k=0}^{j+1}\binom{i+2k}{k}=f(i,j)+\binom{i+2j+2}{j+1}$$

$$\begin{aligned}f(i,j)+f(i+2,j)&=\sum_{k=0}^{j}\binom{i+2k}{k}+\sum_{k=0}^{j}\binom{i+2+2k}{k}\\&=\binom{i}{0}+\binom{i+2j+2}{j}+\sum_{k=1}^{j}\binom{i+2k}{k}+\binom{i+2k}{k-1}\\&=\binom{i+2j+2}{j}+f(i+1,j)\end{aligned}$$

B. 括号序列

上页的公式

$$f(i,j+1) = f(i,j) + \binom{i+2j+2}{j+1}$$

$$f(i,j) + f(i+2,j) = \binom{i+2j+2}{j} + f(i+1,j)$$

解法（续）

令$g(i,j) = [f(i,j), f(i+1,j)]$，根据上页公式，可以发现，可以从$g(i,j)$计算得出$g(i \pm 1,j), g(i,j \pm 1)$，且转移的复杂度是$O(1)$ 的。

因此，f 的$O(n)$ 个点值可以通过分块的方式计算，复杂度为$O(n\sqrt{n})$。

这仍然不能通过本题。

B. 括号序列

需要做什么

在p_T 路径上的某些点(a, b)，计算：

$f(a+1-b, m-a-1)$ 和$f(a+3-b, m-a-2)$

解法（续）

下面考虑$f(a+1-b, m-a-1)$ 这个系列（另一系列同理）：

定义计算$f(i, j)$ 时的势能$w(i, j) = i + 2j + 1$。

沿着路径移动到(a, b) 位置时，计算的f 点值是$f(a+1-b, m-a-1)$，此处势能为$w(a+1-b, m-a-1) = 2m - a - b$。由于：

- 在网格上移动，a, b 都是不减的，则势能也是不减的。
- 移动一步，a 或者b 增加1，势能减少1。
- $(a, b) = (0, 0)$ 的势能为$2n$，$(a, b) = (m, m)$ 的势能为0。

根据上述势能变化规律，对所有需要的f 求点值的复杂度是$O(n)$ 的。

B. 括号序列

解法（总结）

- 先统计所有符合要求的前缀；
- 对于所有右括号的位置，统计前面左右括号的数目a, b；
- 按照$a+b$ 递增的顺序（即按照序列顺序），递推计算所有的点值$f(a+1-b, m-a-1)$ 和$-f(a+3-b, m-a-2)$。
- 求和得到最终答案。

最终解法的复杂度是$O(n)$ 的。

fun fact

如果你写的分块算法忘记排序了，那恰好歪打正着是正解。

J. 献给空白的无败冠冕

题目描述

给定一个$2 \times n$ 的棋盘，第i 行第j 列有$a_{i,j}$ 枚硬币，其中一些位置已经给定，另一些位置为-1。

玩家从$(1,1)$ 走到$(2,n)$，每一步只能向右或向下。由于棋盘只有两行，一条路径等价于选择一个下移列k。

白先走并收集路径上的硬币，空再走并收集自己路径上未被白拿走的硬币。

要求把所有-1 替换为$[1,10^9]$ 内的正整数，使得白唯一选择一条错误路径；若无法构造，输出-1。

数据范围：$1 \le n \le 2 \cdot 10^5$。

出题人：Ruimin Zhang (paulzrm)

J. 献给空白的无败冠冕

观察

真正影响路径选择的格子只有：

$$x_i = a_{1,i+1}, \qquad y_i = a_{2,i} \qquad (1 \le i < n).$$

也就是每一列之间的“斜对角”二元组(x_i, y_i)。
原来的$a_{1,1}$ 和$a_{2,n}$ 会被白的所有路径经过，因此不会影响白选择哪一条路径。

上下界

若格子已经给定，则上下界相等；若为-1，则可取$[1, 10^9]$ 内任意整数：

$$\mathrm{lx}_i \le x_i \le \mathrm{ux}_i, \qquad \mathrm{ly}_i \le y_i \le \mathrm{uy}_i.$$

J. 献给空白的无败冠冕

路径收益

若白选择下移列k，白没有拿走的两段为：

$$U_k = \sum_{i=k}^{n-1} x_i, \quad L_k = \sum_{i=1}^{k-1} y_i.$$

其中U_k 是上方剩余部分，L_k 是下方剩余部分。
空的最优收益为：

$$G_k = \max(U_k, L_k).$$

J. 献给空白的无败冠冕

转化

记所有格子的硬币总和为S。白选择下移列k 时，白拿到的硬币数为：

$$W_k = S - U_k - L_k.$$

所以白最大化W_k，等价于最小化：

$$R_k = U_k + L_k.$$

但是正确目标应该是最小化：

$$G_k = \max(U_k, L_k).$$

核心

要构造一个棋盘，使得R_k 有唯一最小点，但这个点不是G_k 的最小点。

J. 献给空白的无败冠冕

相邻路径的变化

考虑相邻两条路径k 和$k+1$，从k 移到$k+1$ 时：

$$U_{k+1} = U_k - x_k, \quad L_{k+1} = L_k + y_k.$$

因此：

$$R_{k+1} - R_k = y_k - x_k.$$

所以白选择哪条路径，只取决于每一对(x_i, y_i) 的差值。

J. 献给空白的无败冠冕

唯一最小点

若希望白唯一选择下移列p，也就是R_p 是唯一最小值。
对于左侧任意$t < p$，需要$R_t > R_p$，即：

$$\sum_{i=t}^{p-1}(x_i - y_i) > 0.$$

对于右侧任意$t \geq p$，需要$R_{t+1} > R_p$，即：

$$\sum_{i=p}^{t}(y_i - x_i) > 0.$$

也就是说，p 左边所有后缀和要为正，p 右边所有前缀和要为正。

J. 献给空白的无败冠冕

错误路径只需看相邻

假设白唯一选择路径p，但这个选择是错误的。
若$U_p > L_p$，空主要拿的是上方剩余部分。为了让空拿得更少，白应该把下移列向右移动。
若存在某个$q > p$ 使得$G_q < G_p$，那么$p+1$ 已经满足$G_{p+1} < G_p$。
左侧同理。
因此只需要考虑：

- 右侧反例：白唯一选择p，但$p+1$ 更优；
- 左侧反例：白唯一选择p，但$p-1$ 更优。

J. 献给空白的无败冠冕

右侧反例

先考虑右侧反例：白唯一选择p，但$p+1$ 更优。
因为：

$$U_{p+1} = U_p - x_p < U_p, \quad L_{p+1} = L_p + y_p.$$

只要$L_p + y_p < U_p$，就有$G_{p+1} < G_p$。
等价地，右侧反例需要：

$$\sum_{i=p}^{n-1} x_i - y_p > \sum_{i=1}^{p-1} y_i.$$

J. 献给空白的无败冠冕

右侧反例：左半部分

为了让p 左侧所有后缀和为正，需要：

$$\sum_{i=t}^{p-1}(x_i - y_i) > 0 \qquad (t < p).$$

为了让这个条件最容易成立，并让右侧不等式右边尽量小，取：

$$x_i = \mathrm{ux}_i, \qquad y_i = \mathrm{ly}_i \qquad (i < p).$$

令$e_i = \mathrm{ux}_i - \mathrm{ly}_i.$，若$e_1, \ldots, e_{p-1}$ 的所有后缀和都为正，则：

$$\mathrm{leftMinY}_p = \sum_{i=1}^{p-1} \mathrm{ly}_i.$$

J. 献给空白的无败冠冕

快速计算leftMinY

用一次扫描维护“以当前位置结尾的最小后缀和”。
设：

$$s_i = \min_{1 \le t \le i} \sum_{j=t}^{i} e_j.$$

转移为：

$$s_i = e_i + \min(0, s_{i-1}).$$

若$s_{p-1} \ge 1$，则左半部分可行，并且：

$$\mathrm{leftMinY}_p = \sum_{i=1}^{p-1} \mathrm{ly}_i.$$

否则令$\mathrm{leftMinY}_p = +\infty$。

J. 献给空白的无败冠冕

右侧反例：中间位置

中间位置p 需要保证右边严格上升：

$$y_p - x_p > 0.$$

记$z = y_p - x_p$，由上下界得到：

$$\mathrm{ly}_p - \mathrm{ux}_p \le z \le \mathrm{uy}_p - \mathrm{lx}_p.$$

同时必须满足：

$$z \ge 1.$$

为了让右侧反例更容易成立，取满足条件的最小z。

J. 献给空白的无败冠冕

右侧反例：右半部分

对于$i > p$，为了尽量增大右侧不等式左边，取：

$$y_i = \mathrm{uy}_i.$$

令$w_i = x_i - y_i$，则$L_i \le w_i \le U_i$。
其中：

$$L_i = \mathrm{lx}_i - \mathrm{uy}_i, \qquad U_i = \mathrm{ux}_i - \mathrm{uy}_i.$$

J. 献给空白的无败冠冕

右侧前缀限制

右边所有前缀和要为正：

$$z - \sum_{i=p+1}^{t} w_i > 0 \qquad (t > p).$$

等价于：

$$\sum_{i=p+1}^{t} w_i \le z - 1.$$

为了存在合法的w_i，所有前缀的下界和必须不超过$C = z - 1$：

$$\max\left(0, \max_{t>p} \sum_{i=p+1}^{t} L_i\right) \le C.$$

J. 献给空白的无败冠冕

右侧反例：最小合法z

因此z 至少需要满足：

$$z \geq 1 + \max\left(0, \max_{t>p} \sum_{i=p+1}^{t} L_i\right).$$

综合中间位置本身的上下界，取：

$$z = \max\left(1, \mathrm{ly}_p - \mathrm{ux}_p, \mathrm{need}_p + 1\right), \quad \mathrm{need}_p = \max\left(0, \max_{t>p} \sum_{i=p+1}^{t} L_i\right).$$

若：

$$z > \mathrm{uy}_p - \mathrm{lx}_p,$$

则这个p 不可能作为右侧反例中心。

J. 献给空白的无败冠冕

右半部分最大贡献

确定z 后，令$C = z - 1$。
现在要在所有前缀和不超过C 的限制下，最大化：

$$\sum_{i=p+1}^{n-1} w_i.$$

若全部取上界，则贡献为：

$$\sum_{i=p+1}^{n-1} U_i.$$

若某个前缀被C 卡住，则该前缀最多为C，后缀继续取上界。

J. 献给空白的无败冠冕

右半部分最大贡献（续）

因此最大值为：

$$\text{best}_p = \min\left(\sum_{i=p+1}^{n-1} U_i,\ C + \min_{q \geq p+2} \sum_{i=q}^{n-1} U_i\right).$$

如果$p = n - 1$，右半部分为空，此时：

$$\text{best}_p = 0.$$

这些值可以用U_i 的后缀和以及后缀最小值$O(1)$ 得到。

J. 献给空白的无败冠冕

右侧反例的判定

右侧不等式为：

$$\sum_{i=p}^{n-1} x_i - y_p > \sum_{i=1}^{p-1} y_i.$$

左边最大值为：

$$\mathrm{maxRight}_p = -z + \sum_{i=p+1}^{n-1} \mathrm{uy}_i + \mathrm{best}_p.$$

右边最小值为$\mathrm{leftMinY}_p$。
所以右侧反例存在当且仅当：

$$\mathrm{leftMinY}_p < +\infty \quad \text{且} \quad \mathrm{maxRight}_p > \mathrm{leftMinY}_p.$$

J. 献给空白的无败冠冕

右侧反例的预处理

为了$O(1)$ 判断每个p，预处理：

- ly_i 的前缀和；
- $e_i = \mathrm{ux}_i - \mathrm{ly}_i$ 的最小后缀和；
- $L_i = \mathrm{lx}_i - \mathrm{uy}_i$ 的最大前缀和；
- $U_i = \mathrm{ux}_i - \mathrm{uy}_i$ 的后缀和与后缀最小值；
- uy_i 的前缀和。

这样所有p 的右侧反例判定总复杂度是$O(n)$。左侧反例同理。

J. 献给空白的无败冠冕

构造右侧反例

若找到右侧反例中心p，按判定过程构造。
首先对$i < p$:

$$x_i = \mathrm{ux}_i, \qquad y_i = \mathrm{ly}_i.$$

然后选择判定时的最小合法z，在第p 对中取一组满足:

$$y_p - x_p = z$$

且在上下界内的(x_p, y_p)。
最后对$i > p$，取:

$$y_i = \mathrm{uy}_i, \qquad x_i = y_i + w_i.$$

J. 献给空白的无败冠冕

构造w_i

需要构造w_i，满足：

$$L_i \le w_i \le U_i,$$

且所有前缀和不超过：

$$C = z - 1.$$

做法是：

1. 先计算每个前缀和的最小可能值lo；
2. 再计算每个前缀和的最大可能值hi，并用C 截断；
3. 从最终前缀和hi_m 倒推每个w_i。

J. 献给空白的无败冠冕

倒推过程

设当前前缀和为cur，正在倒推第t 个变量。
上一个前缀和pre 必须同时满足：

$$lo_{t-1} \le pre \le hi_{t-1}, \quad cur - U_t \le pre \le cur - L_t.$$

从交集中任选一个合法pre，然后令：

$$w_t = cur - pre.$$

再继续倒推：

$$cur = pre.$$

因为判定阶段已经保证可行，所以倒推一定可以构造出合法序列。左侧反例同理。

J. 献给空白的无败冠冕

还原答案

构造出的变量满足：

$$x_i = a_{1,i+1}, \qquad y_i = a_{2,i}.$$

所以还原为：

$$a_{1,i+1} = x_i, \qquad a_{2,i} = y_i.$$

没有被设置到的$a_{1,1}$ 和$a_{2,n}$ 不影响决策，若原来为-1，直接赋值为1；否则保留原值。
如果右侧反例和左侧反例都找不到，则输出-1。
总时间复杂度为$O(n)$，空间复杂度为$O(n)$。

fun fact

本题由CF1555C Coin Rows 错解改编而来。